

9-Agent 内容流水线

实现说明 / Implementation Reference

项目: ch-agents · 内容中台

生成于 2026/4/27 09:43:05

9-Agent 内容流水线 · 实现说明

项目位置: apps/api/src/workers/ 队列:BullMQ on Redis,9 条独立队列,worker 在 apps/api/src/workers/index.ts 启动 状态机: raw_items → INGESTED → CLASSIFIED → TITLED → COVERED → COMPLIANCE_PASS / FAIL / REVIEW → PUBLISHED → DISTRIBUTED 审计:每个 agent 调用都包在 withRun() 里,在 agent_runs 表落一行(latency / cost / token / error)

总览

#	Agent	输入状态	输出状态	用 LLM	并发	触发方式
1	source-scoring	—	—	✗ 纯 SQL	1	cron 每天 03:15 + 工作台「执行」
2	ingestion	—	INGESTED	✗	6	cron 每小时 + auto-pipeline 每 15s
3	classification	INGESTED	CLASSIFIED	✓ Haiku(可关)	1	上游持久化后入队
4	title	CLASSIFIED	TITLED	✓ Sonnet(可关)	1	同上
5	cover	TITLED	COVERED	✓ Haiku(只生成短语)	6	同上
6	compliance	COVERED	COMPLIANCE_PASS / FAIL / REVIEW	✓ Sonnet(L1命中跳过)	1	同上
7	publishing	COMPLIANCE_PASS	PUBLISHED	✗	2	人工门控,需 UI 批准
8	distribution	PUBLISHED	DISTRIBUTED	✓ Sonnet	3	人工门控,UI 批准生成文案
9	analytics	—	写 analytics_daily	✗	1	cron 每天 02:00

auto-pipeline 每 15s tick 一次,凡是设了「⚡ 自动」的非门控 agent 就批量入队它的待处理项;门控 agent(publishing / distribution)永远不自动触发。

1. Source Scoring · 源评分

文件:[apps/api/src/workers/source-scoring/scorer.ts](#)

目的:周期性给每个采集源打分,决定优先级、风险等级、是否拉黑/暂停。

实现:纯 SQL,零 LLM,零外部调用。一条 CTE 把每个源的指标聚合后,在 Node 里加权:

```
score = 50
+ 已发布数 × 2
+ 新鲜度(2 天内 +10 / 7 天内 0 / 更老 -10)
+ 7 天活跃度(min 15)
+ PV 阶梯(>1w +10 / >1k +6 / >100 +3)
+ 收入阶梯(>$10 +5 / >$1 +3)
- 合规 FAIL 数 × 5
- 合规 REVIEW 数 × 2
```

输出:写回 `sources.score / risk_level / stability / status`:

- `risk_level`: 5+ FAIL → high, 2+ FAIL 或 5+ REVIEW → medium, 否则 low
- `status`: 20+ FAIL → blacklist, `score < 20` → paused, 否则 active

触发: `scheduler.ts` 每天 03:15 跑一次,工作台也可手动触发。

2. Ingestion · 内容采集

文件:[apps/api/src/workers/ingestion/index.ts](#)

目的:从配置的源拉原始内容、清洗、去重、入库,触发后续分类。

两层任务:

- `kind: 'fanout'` — 遍历所有 active 源,按 `grayscale_pct` 概率筛选后给每个源各发 1 个 ingest 任务
- `kind: 'ingest', sourceId` — 单源采集

单源流程:

```
1. 查 sources WHERE id=... AND status='active'
2. adapter = adapterFor(source.platform)           ← rss / html / 2ksg / knit
3. candidates = await adapter.fetch(source)         ← 一次性拿到所有候选
4. for c in candidates:
  a. 用 dedupe_key 查 URL 级去重 → dupUrl
  b. 短文本走 c.text;长文本走 readability(clean.ts)
  c. 算 sha256(normalized) + simhash(64bit)
  d. simhash 距离 ≤3 → dupContent(per-image 模式跳过此步)
  e. INSERT raw_items + items(status=INGESTED) 一个事务
  f. enqueue classification 任务(jobId=classify__<itemId>)
```

5. 返回 `IngestStats:{candidates, ingested, dupUrl, dupContent, cleanFail, errors, authFail?}`

Adapter 注册表([adapters/index.ts](#)):

platform	用途	关键文件
rss	RSS / Atom	rss.ts
html	固定 URL 列表(article / per-image / crawl)	html.ts
2ksg	图册 JSON API(自动探测 bucket + xo-hex 解码)	json-2ksg.ts
knit	Cloudflare 编号图册(URL 模板推断 + cookie 注入)	knit.ts

鉴权失败检测:adapter 在请求被 403/503/全 bucket reject 时抛 `AdapterAuthError`,worker 捕获后在 `agent_runs.output` 写 `authFail: true`,工作台顶部红条提示去刷 cookie/token。

触发:cron 每小时整点 + auto-pipeline 每 15s + 工作台手动。worker 并发 6。

3. Classification · 分类打标

文件:[apps/api/src/workers/classification/](#)

目的:给 item 打分类、标签、关键词。

两条路径:

- `CLASSIFICATION_SKIP_LLM=1`:走 `categorize.ts:classifyByRules`。15 个固定分类(AI / 硬件 / 软件工程 / 网络安全 / ...)对应正则关键词组,文本里命中数最多的分类胜出,标签从命中词 + 标题分词中取。
- 默认:Haiku tool-use 调用 `classify`。`TAXONOMY_PROMPT` 走 prompt cache,温度 0,工具 schema 强制返回 `{category, tags[], keywords[]}`。

入库: items SET category, tags, keywords, status='CLASSIFIED',然后入队 title。

4. Title · 标题生成

文件:[apps/api/src/workers/title/](#)

目的:生成 SEO 友好的标题、摘要、URL slug。

两条路径:

- `TITLE_SKIP_LLM=1`:用原始抓取标题作单一候选,正文前 180 字作摘要。
- 默认:Sonnet + tool-use 一次出 3 个不同风格的候选 + `best_index` + 摘要。System prompt 要求:候选 ≤30 字、风格分层(事实型/观点型/结果型)、摘要 120-180 字、英文原文翻译改写不直译。

slug:[slug.ts:makeUniqueSlug](#) 把中文 → pinyin/拉丁化 → 唯一性校验(items_slug_key 唯一索引)。

入库: title / summary / slug / title_version++ / status='TITLED', 入队 cover。

5. Cover · 封面与图集

文件:[apps/api/src/workers/cover/](#)

目的:从 raw_items.media_urls 选/裁/上传封面图;HTML-list 类型的源还会留前 N 张作为图集。

流程:

1. 拉 media_urls(最多 20 张)
2. 每张:axios 下载(限 6MB)→ sharp 读元数据 → 评分
score = width × height × aspectPenalty
width<400 或 height<260 直接淘汰
aspect >3 或 <0.5 给 0.25 倍惩罚
3. 排序取 top-N(html 平台 N=6, 其它 N=1)
4. 每张图渲染 3 个尺寸:card 600×400 / og 1200×630 / thumb 240×160
sharp resize fit=cover position=attention,JPEG q=82 mozjpeg
5. 上传 MinIO:covers/<itemId>/<size>.jpg(主图)+ covers/gallery/<i>/<itemId>/<size>.jpg(图集)
6. (可选)Haiku 生成 ≤14 字封面短语 cover_copy
7. UPDATE items SET cover_url=card 尺寸URL, cover_sizes=JSON 全部尺寸 + gallery, cover_copy
8. 入队 compliance

LLM:仅用于封面文案,写不出来不阻塞流水线。

6. Compliance · 合规审核

文件:[apps/api/src/workers/compliance/](#)

目的:阻拦违规内容,介于 PASS / REVIEW(人工)/ FAIL 三种结局。

两层(L1 + L2):

L1 黑名单([blacklist.ts](#)):3 大类正则,命中即直接 FAIL,不调 LLM。

- `politics_sensitive` (占位,需按你的合规要求填)
- `nsfw_explicit`
- `copyright_watermark` (版权水印 / Shutterstock / Getty 文字)

L2 LLM 评分([score.ts](#)):L1 没命中时,Sonnet 对 6 个维度各打 0-3 分:

- 政治敏感 / 色情低俗 / 暴力恐怖 / 版权争议 / 医疗夸大 / 金融诱导

决策([decide.ts](#)):

条件	状态	trigger
L1 命中	COMPLIANCE_FAIL	blacklist
L2 任一维度 ≥3	COMPLIANCE_FAIL	llm_reject
L2 任一维度 ≥2	COMPLIANCE_REVIEW (人工待定)	llm_review
L2 全 <2	COMPLIANCE_PASS → 入队 publishing	pass
L2 失败/未跑	COMPLIANCE_REVIEW	llm_review

入库: items SET status, compliance_status, risk_tags, compliance_reasons (reasons 是 JSON, 带触发原因 + 各维度分数 + L1 命中详情)。

人工门控:REVIEW 状态出现在工作台合规 agent 行下方,人工点「✓ 批准」(进 PASS)或「✗ 拒绝」(进 FAIL)。

7. Publishing · 发布

文件:[apps/api/src/workers/publishing/index.ts](#)

目的:把 PASS 的 item 标记为 PUBLISHED,触发 Next.js ISR 重新验证页面。

流程:

```
1. 查 items.status,严格模式(PUBLISH_STRICT=1)只接受 COMPLIANCE_PASS / PUBLISHED;宽松模式什么状态都接
2. 没有 slug 就用 title 现场生成一个
3. UPDATE items SET status='PUBLISHED', slug, published_url='/a/<slug>', published_at=NOW()
4. POST 到 web 应用的 /api/revalidate(密钥头 x-revalidate-secret),让以下页面立即可访问:
  - /a/<slug>
  - /
  - /category/<urlencoded>
  - /sitemap.xml
```

关键点:不发到任何外部站点。文章只出现在本项目的 Next.js 站点(localhost:3000 或 SITE_URL),路由 /a/<slug> 由 [apps/web/src/app/a/[slug]/page.tsx](../apps/web/src/app/a/[slug]/page.tsx) 渲染。

人工门控: auto: false 默认,工作台「发布」tab 列出待发文章,人工勾选 → 「批准发布」入队。

8. Distribution · 分发

文件:[apps/api/src/workers/distribution/](#)

目的:为外部社交平台改写文案。

当前实现:Sonnet 改写为 Twitter/X 推文(280 字符以内、2-3 个 hashtag、带文章链接)。

流程:

1. 查 items WHERE status='PUBLISHED'(并去重避免同一 item × 同一 channel 重复)
2. Sonnet 改写得到 copy
3. INSERT distribution_tasks (item_id, channel, copy, status='pending')
4. UPDATE items SET status='DISTRIBUTED'

关键点:不调 Twitter API,只生成文案写到 `distribution_tasks`。工作台「分发」tab 把每条 task 列出来 + 「复制文案」按钮,人工 copy → 去 Twitter/X 粘贴 → 回工作台点「✓ 已发出」标记 status=sent。

为什么不自动发:各平台官方 API 对自动发文限制极严(微信公众号要审核账号、小红书无公开 API、Twitter 需要付费 API),手动复制粘贴是当前最务实的折中。

要扩展其它渠道:在 `rewrite.ts` 里加 system prompt + DistributionItem 类型,在 `admin-pipeline.ts` 暴露选择 channel 的入口。

9. Analytics · 数据回流

文件:[apps/api/src/workers/analytics/](#)

目的:T+1 拉取 GA4 流量数据,作为 source-scoring 的输入信号。

两个 job kind:

`pull` ([ga4.ts:pullAnalyticsYesterday](#)):

1. 查 GA4 Data API:dimension=pagePath BEGINS_WITH '/a/',metrics={PV, UV, avg_session_duration}
2. 把 pagePath 解析成 slug → 查 items 表拿 item_id
3. UPSERT analytics_daily (item_id, date, channel='site', pv, uv, avg_duration)

降级:没设 `GA4_PROPERTY_ID` 或没装 `@google-analytics/data` 时直接 noop 返回,不阻塞流水线。

`report` ([report.ts](#)):生成周报(top items / 分类 / 来源)。

触发:cron 每天 02:00 UTC(等昨日数据 settle)。

回路闭环:analytics_daily.pv / revenue → source-scoring SQL → 高 PV 源加分 → ingestion fanout 优先 → 高质量内容更多。

跨 agent 共享基建

withRun() (packages/agents/src/runner.ts)

```
withRun({ agent: 'classification', itemId }, async () => {  
  return { output, model, usage, cost }  
})
```

插入 agent_runs 行(running),回调跑完 UPDATE 成 success(带 latency / token / cost),失败 UPDATE 成 failed(带 error 前 2000 字)。所有 agent 强制走这个,工作台的成本统计、运行历史、auth-fail 检测全靠这张表。

callClaude() (packages/agents/src/anthropic.ts)

Anthropic SDK 薄封装:

- 模型名简写: haiku / sonnet / opus → 实际 model id
- prompt cache(system: [{text, cache:true}])
- 自动算 cost: input + cached + output × 各档单价,返回 costUsd
- 重试 3 次(指数退避),429 / 5xx 自动重试

BullMQ 队列(packages/agents/src/queue.ts)

9 个队列名常量在 QUEUE_NAMES。worker 启动用 startWorker(name, handler, {concurrency}), job ID 模式 <agent>__<itemId> 自动去重(同一 item 并发重入只跑一次)。

Auto-pipeline(apps/api/src/auto-pipeline.ts)

每 15s tick:

- 跳过 globalStop 状态
- 跳过 paused 或 auto: false 的 agent
- 跳过 publishing / distribution 这两个硬人工门控(即使设了 auto 也不触发)
- 其它 agent 查待处理 SQL 计数,有的话批量入队(每次 LIMIT 200 防雪崩)

灰度采样

sources.grayscale_pct (0-100)在 fanout 时按概率筛选源。设 30 表示该源 30% 概率被本轮采集 — 用来给老/不稳定的源降权。